

*RenderSurfaceLighting()*

**for all** lights **do**

    Clear Depth Texture

**for all** surfaces **do**

        Generate MVP

*WriteDepthBuffer()*

**end for**

**end for**

*WriteDepthBuffer()*

Set shader program *patch\_tess\_pass*

Assign SSBO's and uniforms, light\_pass = 1

**for all** surface control points (concurrently) **do**

    Dispatch compute shader *patch\_tess\_pass*

    Get a triangulated version of our patch control points as an EBO

**end for**

Set shader program *patch\_shadow*

Render to depth buffer the triangulation of the control points

**if** visualize **then**

    Set shader program *visualize\_tiles*

    Render triangulated tiles to primary camera

**end if**

*patch\_tess\_pass: light pass*

Interpret raw control point vertex data

Calculate 2nd derivative in the U direction

Calculate 2nd derivative in the V direction

**for all** tiles **do**

    Get indices of its two triangles

    Place those six indices into the EBO with control points as vertices

    (triangulating shadow mapping)

**end for**

```

patch_shadow
for all vertices do
    Project from light source
    Save projection of control point triangulation to light's depth buffer
end for

```

```

RenderSurfaces()
for all surfaces do
    UpdatePatchTessLevels()
    RenderSurface()
end for

```

```

UpdatePatchTessLevels()
Set shader program patch_tess_pass
Assign SSBO's and uniforms, light_pass = 0
for all surface control points (concurrently) do
    Dispatch compute shader patch_tess_pass
end for

```

```

patch_tess_pass: tess pass
Interpret raw control point vertex data
Calculate 2nd derivative in the U direction
Calculate 2nd derivative in the V direction
for all patches do
    for all 9 bi-1 quads of a bi-3 BB-net, simultaneously do
        Compute center box
    end for
    Get corner of avg center box closest to the camera to get minimal
    screen-coordinate-aligned rectangle that covers slefe box
    Apply minimal tessellation factor that achieves parametric-accuracy
    if patch is entirely out of view then
        Set tessellation factor to 0.0
    end if
end for

```

*RenderSurface()*

Set shader program *render\_pass*

Assign SSBO's and uniforms

Draw Elements (render\_pass.vs, .tcs, .tes, .fs)

\\IMPORTANT

\\Tessellation factors are assigned in Tess Control shader

\\Fragment shader reads depth buffer to create shadows